

PART VII — PRODUCTION, DEPLOYMENT, AND BEYOND

Chapter 37

Evaluation Frameworks for RAG

“A retrieval pipeline that has never been measured against anything is not reliable — it is merely unexamined.”

Chapter 37 — Evaluation Frameworks for RAG

The honest answer, worked out across this chapter's five sections, is that this project never adopted a packaged RAG evaluation framework. It built three narrow, purpose-specific tools instead — an inline LLM-as-judge layer, a production-log parser, and a feature-flag ablation harness — and relied on their combination rather than on any single scoring library. That choice is itself instructive: understanding **why** a team with real production traffic and a real self-learning layer chose triangulation over a packaged framework is more useful than a survey of framework feature lists.

37.1 Retrieval metrics — recall@k, precision@k, MRR, nDCG

Definition — Recall@k / Precision@k / MRR / nDCG

Recall@k is the fraction of all truly relevant documents that appear somewhere in the top k retrieved results. Precision@k is the fraction of the top k results that are actually relevant. Mean Reciprocal Rank (MRR) scores the position of the first relevant result — $1/\text{rank}$, averaged across queries. Normalized Discounted Cumulative Gain (nDCG) extends this to graded relevance, discounting a relevant result's contribution the further down the ranked list it appears.

These four numbers are the standard vocabulary of information-retrieval evaluation, and every one of them shares the same hard requirement: a labeled ground truth, a fixed set of queries each paired with a known-correct set of relevant documents, established independently of whatever system is being scored. Recall@k and precision@k tell you, respectively, whether the system found the right things and whether what it found was mostly right; MRR and nDCG go further and reward ranking the best result near the top rather than merely including it somewhere in the list.

This project's retrieval pipeline (Chapter 11) has never had such a labeled set to score against, and its research record is explicit about why: Research topic 8 evaluates BM25 hybrid scoring, Maximal Marginal Relevance for diversity, and cross-encoder reranking as retrieval-ranking upgrades, and states plainly that none of them is implemented — "current pipeline uses cosine similarity only." Cosine similarity against an embedding index is a ranking signal, not a metric; it produces an ordering, but nothing in this project computed recall@k or nDCG against it, because doing so would require a curated set of query/relevant-document pairs this project never built. Research topic 52's gap analysis against LangSmith names this directly: a curated evaluation dataset separate from production logs is listed as a missing piece, not a completed one.

That gap is worth sitting with rather than glossing over, because it is common, not unusual. A curated retrieval-evaluation set has to be built by someone reading real queries and real chunks and manually labeling relevance — expensive, slow to keep current as a corpus grows, and easy to defer indefinitely in favor of shipping the next feature. Section 37.4 shows what this project measured instead of recall@k, and it is worth previewing here: this project's ``validate_document_retrieval`/`validate_learned_qa_retrieval`` judges (Chapter 11) score relevance per-chunk, per-request, in production, which is a very different and much noisier signal than recall@k against a fixed labeled set — but it is a signal that actually existed and actually ran on every real query, where the formal metric never did.

37.2 Generation metrics — faithfulness, answer relevancy, context precision

Definition — Faithfulness / Answer relevancy / Context precision

Faithfulness measures whether every claim in a generated answer is actually supported by the retrieved context (the opposite of hallucination). Answer relevancy measures whether the answer addresses the question asked, independent of whether it's grounded. Context precision measures whether the retrieved context that was actually used to generate the answer was relevant, as opposed to noise the model had to filter past.

These three names come from the RAGAS literature (Section 37.3), but this project implements something that maps closely onto faithfulness and answer relevancy without ever naming them that way. `check_answer_quality()` in `app_workflow/nodes/check_answer_quality.py` runs the `'GROUNDING_PROMPT'` against every synthesized answer and classifies it into one of five verdicts rather than a single faithfulness score: `'GROUNDED'`, `'PARTIALLY_FABRICATED'`, `'OVERCLAIMED'`, `'OFF_TOPIC'`, or `'UNKNOWN'`. The prompt's own internal structure makes the mapping explicit — RULE 1 checks sentence-by-sentence fabrication (faithfulness), RULE 2 checks topical relevance (answer relevancy), and RULE 3 checks completeness and calibration against what the retrieved context actually supports.

A five-way verdict taxonomy is a deliberately richer design than a single pass/fail faithfulness score, and the reason is operational rather than academic: each of the five verdicts implies a different downstream action. `'PARTIALLY_FABRICATED'` and `'OVERCLAIMED'` both indicate a faithfulness problem, but they are different failures — the first invents content the context never supported, the second overstates confidence in content the context only weakly supports — and a system that only ever produced a binary FAIL would conflate them, along with the genuinely different `'OFF_TOPIC'` failure (Section 37.1's context-precision idea in miniature: the model answered *something*, just not the question asked) and the honest `'UNKNOWN'` case, where the judge itself couldn't confidently classify the answer at all. Figure 37.1 makes this collapse concrete.

A binary PASS/FAIL hides which of five distinct failures actually happened

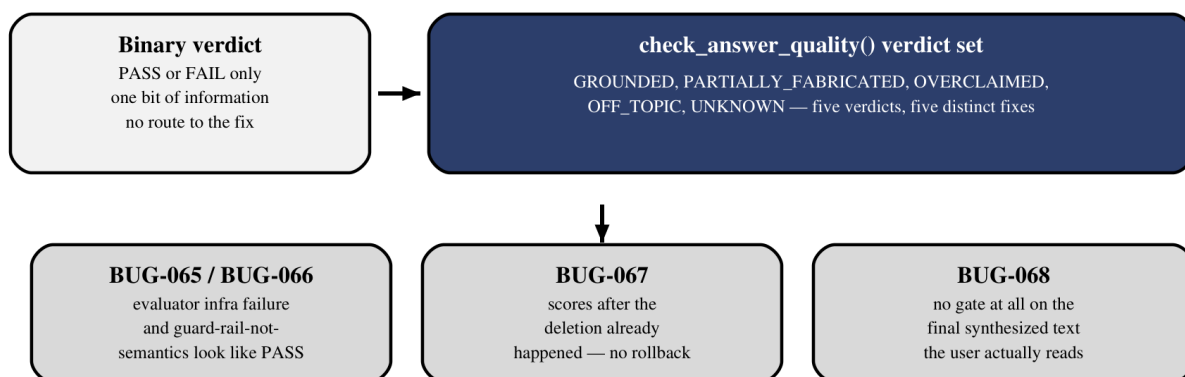


Figure 37.1 — Collapsing five distinct generation failures into one PASS/FAIL bit destroys the information a fix needs; three real bugs (065, 066, 067) show what a coarser gate would have missed.

The same faithfulness-adjacent judging pattern appears earlier in the pipeline, not just at final answer synthesis. ``validate_merge``, ``validate_redundancy``, and ``validate_lbc`` (all in ``app_workflow/services/validators.py``) apply structurally similar LLM-as-judge checks to merge faithfulness, deduplication redundancy, and compression faithfulness respectively — each one is, in effect, a context-precision or faithfulness check scoped to one intermediate pipeline stage rather than the final answer. Chapter 22B's semantic compression and Chapter 34's chunk deduplication both depend on exactly this pattern of judge-per-stage rather than judge-only-at-the-end, precisely because a faithfulness failure introduced during compression is much cheaper to catch immediately than to trace back to from a bad final answer.

37.3 RAGAS, TruLens, DeepEval — tools of the trade

Definition — RAGAS / TruLens / DeepEval

RAGAS is an open-source library purpose-built for RAG evaluation, computing faithfulness, answer relevancy, and context precision/recall as LLM-judged scores against a reference dataset. TruLens instruments a running application to compute its own "RAG Triad" (context relevance, groundedness, answer relevance) plus general LLM-app feedback functions, with a dashboard for cross-run comparison. DeepEval is a pytest-style testing framework that expresses evaluation metrics as assertions, making RAG evaluation runnable inside a normal CI test suite.

None of these three appears anywhere in this project's codebase or its research ledger — a direct search across ``docs/Research.md`` and the full Memora source tree turns up zero references to RAGAS, TruLens, or DeepEval by name. That absence is worth stating plainly rather than working around, because the honest reason is instructive: this project's evaluation research effort went almost entirely toward LLM-application **observability** platforms — Phoenix, LangSmith, and Langfuse (Chapter 38) — rather than toward RAG-specific scoring libraries layered on top of them.

The three tools are not redundant with each other, and knowing the difference matters when a team is choosing among them for a project that, unlike this one, decides to adopt one. RAGAS is metric-first: it assumes you already have a reference dataset and want standardized faithfulness/relevancy/precision numbers out of it, and its scores are the closest published equivalent to the five-way verdict ``check_answer_quality()`` produces internally. TruLens is instrumentation-first: it wraps a live application and computes its RAG Triad continuously, closer in spirit to this project's inline ``validate_*`` judges than to a one-time offline scoring run. DeepEval is workflow-first: it exists to make evaluation metrics assertable inside ``pytest``, so that a faithfulness regression fails a CI build the same way a broken unit test would — a discipline this project never built for its LLM-judge layer, since none of ``validate_retrieval``, ``check_answer_quality``, or the others runs as part of an automated test gate today.

The clearest real analog to "should we adopt a packaged evaluation framework" reasoning this project actually did is documented in Research topics 52 and 54, comparing LangSmith, Phoenix, and Langfuse rather than RAGAS/TruLens/DeepEval specifically. Topic 58's proposed (but never implemented) two-level Langfuse scoring scheme — trace-level scores like ``answer_quality``, ``groundedness``, and ``coverage``, plus observation-level scores like ``retrieval_relevance`` and ``compression_faithfulness`` — is the closest this project ever got to sketching a RAGAS-shaped metric taxonomy, and it is worth naming explicitly that it remained a planning document: "None of Scores, Evaluators, Human Annotation, or Datasets are populated yet ... this is planning/reference material for a future evaluation-harness effort," per the research record. A reader adopting RAGAS, TruLens, or DeepEval on a new project is doing, formally, exactly what that unfinished plan sketched.

Common pitfall — Treating a packaged framework as a substitute for domain judgment

RAGAS's context-precision score and this project's `validate_retrieval` judge both ultimately depend on an LLM deciding what counts as "relevant" — a packaged framework does not remove that judgment call, it only standardizes how the judgment gets asked and reported. Adopting RAGAS without first deciding, the way `check_answer_quality`'s `GROUNDING_PROMPT` does, exactly what counts as fabrication versus overclaiming versus off-topic for your own domain just moves the ambiguity into a black box with a more official-looking number attached to it.

37.4 Building a lightweight in-house evaluator

This project's actual evaluation approach, assembled piece by piece across earlier chapters rather than designed as a single system, is a triangulation of three narrow tools, none of which alone constitutes a full evaluator. Figure 37.2 lays the three out side by side.

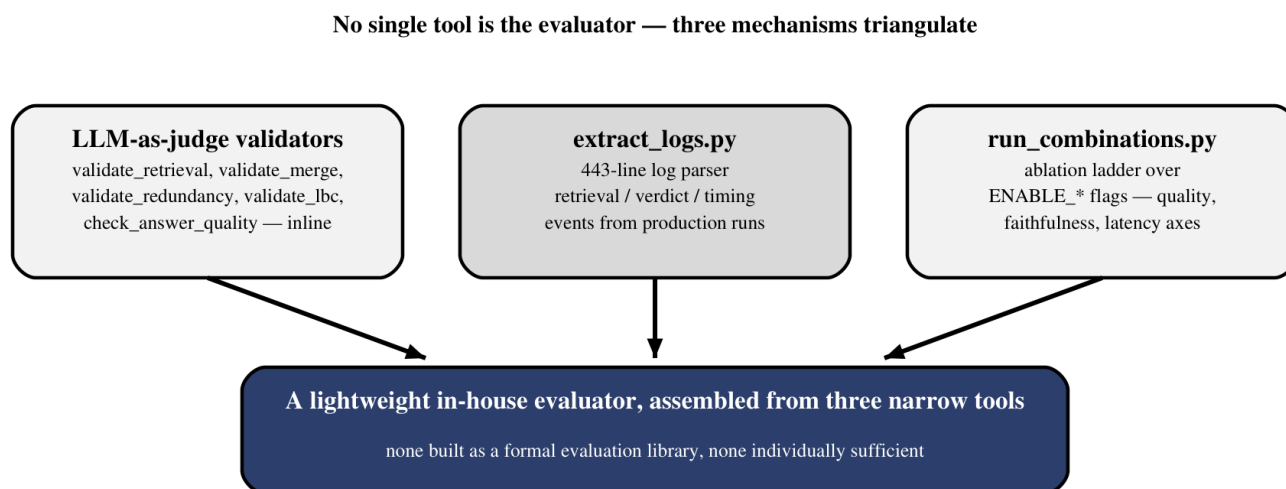


Figure 37.2 — Three narrow, purpose-built tools stand in for a packaged evaluation framework; each covers a gap the other two leave open.

The first leg is the inline LLM-as-judge layer this chapter has already covered — `validate_document_retrieval`, `validate_learned_qa_retrieval`, `validate_merge`, `validate_redundancy`, `validate_lbc`, and `check_answer_quality`. What makes this leg unusual compared to a typical offline evaluation harness is that it runs on every live production request and its verdicts change what the pipeline does next — a FAIL retrieval verdict triggers a retry, a failed redundancy check blocks a deletion. Research topic 58 names this distinction precisely, describing these as "control-plane judges ... which do alter routing," in contrast to the shadow judges a packaged framework like RAGAS or TruLens would run offline, after the fact, with no power to change the request they're scoring.

`app_workflow/services/validators.py` defines the structured shape every one of these judges returns, and the shape itself is worth reading closely — it is a per-chunk evidence array with a required reason, not a bare verdict string:

```
class RetrievalCheckResult(TypedDict):
    verdict: str          # "PASS" | "PARTIAL" | "FAIL"
    per_chunk: list[dict] # [{"chunk_id": ..., "relevant": bool, "reason":
                           str}, ...]
```

Chapter 38.19's still-open BUG-035 is the direct cautionary tale for why this per-chunk array exists at all rather than a single verdict field: a validator's own top-level `verdict` has been observed disagreeing with its own `per\_chunk` evidence in the same response, which means an evaluator that only reads `verdict` and never inspects `per\_chunk` is trusting exactly the field this project's own bug ledger already caught lying.

The second leg is `extract\_logs.py`, the 443-line log parser Chapter 36C's A/B comparison methodology depends on. It turns the unstructured text of `app\_workflow/run\_logs/` debug output into structured retrieval, verdict, and timing events — effectively converting every production request into a lightweight evaluation record after the fact, without requiring a dedicated logging schema built in advance. This is the closest thing this project has to an offline evaluation dataset: not a curated set of labeled queries, but the accumulated trace of everything the system was actually asked in production.

The third leg is `run\_combinations.py`, the ablation harness first introduced in Research topic 20 and exercised across ten scripted runs in the 2026-06-09 development session. Rather than scoring a fixed output against a fixed metric, it evaluates *combinations of feature flags* against three axes — answer quality, faithfulness, and latency — turning the `ENABLE\_\*` switch catalogue from Chapter 36B into the evaluation's independent variable. This is precisely how BUG-020 and BUG-021 were discovered: not by a metric crossing a threshold, but by specific flag combinations crashing or silently doing nothing during a systematic sweep.

None of these three tools alone is a RAG evaluator in the RAGAS sense. The `validate\_\*` judges score individual live requests but never aggregate into a dataset-level number. `extract\_logs.py` structures data but computes no metric of its own — it is preprocessing for the A/B comparison a human still reads and judges. `run\_combinations.py` evaluates configuration space, not answer quality directly. A lightweight in-house evaluator, built this way, is not a single artifact a team designs up front — it accretes, tool by tool, out of whatever the project already needed for another reason, and the discipline that makes the accretion actually useful is remembering to run all three together rather than trusting any one of them in isolation.

37.5 Human evaluation: when automation isn't enough

Every automated judge this chapter has covered is itself an LLM call, and an LLM call judging another LLM call's output inherits a structural blind spot: a failure mode neither the generator nor the judge model was ever trained to recognize as a failure will sail through both unnoticed. This project hit that limit directly and responded with a manual audit rather than a better prompt — Research topic 51's quality audit, conducted by a human reading real production debug traces (with graphify and parallel subagents assisting the reading, not the judging) rather than by any `validate\_\*` function.

That audit scored real traces on five axes — output-structure malformation, hallucination, instruction-following, task completion, and relevance — the same conceptual territory `check\_answer\_quality`'s five verdicts cover, but scored by a human specifically because the automated judges' own reliability was the thing under question. An LLM-as-judge system cannot audit itself for the ways it systematically fails; only a human reading the raw trace, unmediated by another model's summary of it, can catch a judge that has learned to rubber-stamp a familiar-looking wrong answer.

BUG-065 is the concrete case that makes this limit vivid rather than theoretical. When the Hugging Face inference router returned HTTP 402 errors under load, the `judge\_llm` calls behind several `validate\_\*`

functions silently failed and defaulted to an 'UNKNOWN' verdict — and in the affected production log, most of what looked like "clean" verdicts were actually judges that never ran at all, not judges that genuinely approved the content. An automated dashboard tracking the *rate* of PASS verdicts would have shown a healthy-looking trend line for exactly the window where evaluation coverage had silently collapsed to near zero; only a human reading the actual log lines, the way Research topic 51's audit did, caught that the judges themselves were absent rather than agreeing.

The practical rule this leaves a reader with is not "replace automation with humans" — the `validate_*` layer runs on every request precisely because a human cannot review every request — but "budget periodic human review of the judges, not just of the system the judges are watching." A human audit does not need to run on every request the way a control-plane validator does; it needs to run often enough, and read raw enough evidence, to catch the specific failure an automated judge is structurally unable to catch about itself: that it has quietly stopped judging at all, or has learned to approve a category of wrong answer the way BUG-066's length-ratio guard let fabricated citation text through untouched.

Common pitfall — Auditing the system but never auditing the judges

BUG-067 is the sharpest version of this trap: `check_answer_quality`'s successor validator for chunk deletion scored redundancy *after* the deletion had already happened, with no rollback path — meaning a bad verdict didn't just fail to catch a problem, it had no way to undo one it caused. A validator's placement in the pipeline is itself part of what a human audit needs to check; a technically correct verdict that arrives after the damage is already committed evaluates nothing that still matters.

Part VII continues from here into the platforms this chapter's judges and logs actually run on top of — Phoenix, LangSmith, and Langfuse — and the debugging techniques that make a production trace legible enough for either an automated judge or a human auditor to read in the first place.